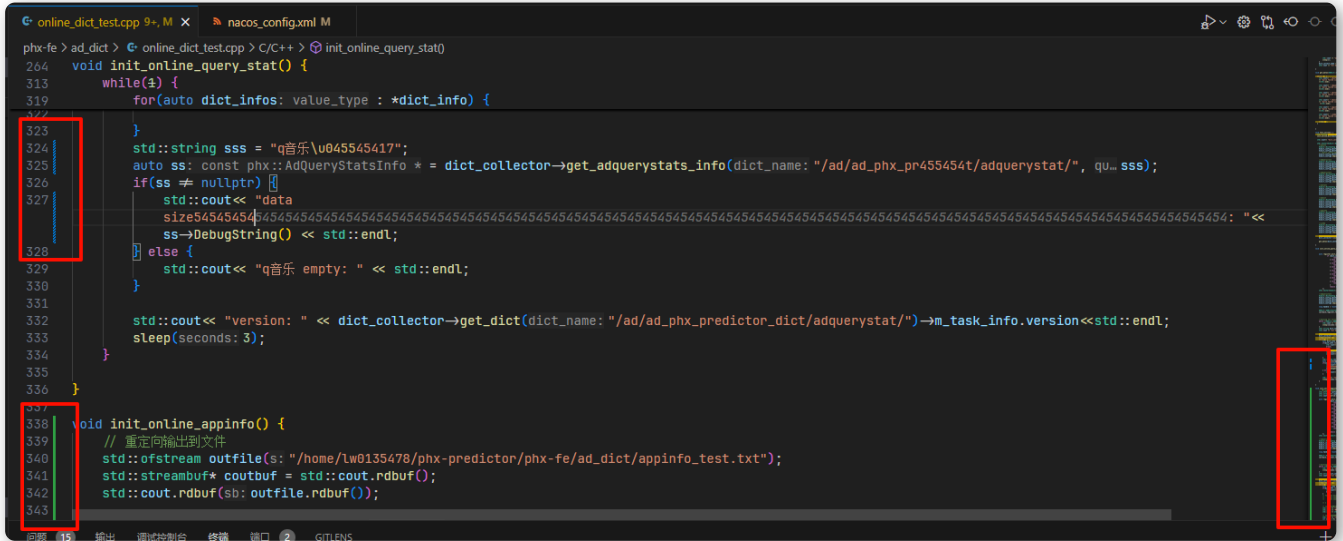


基本操作

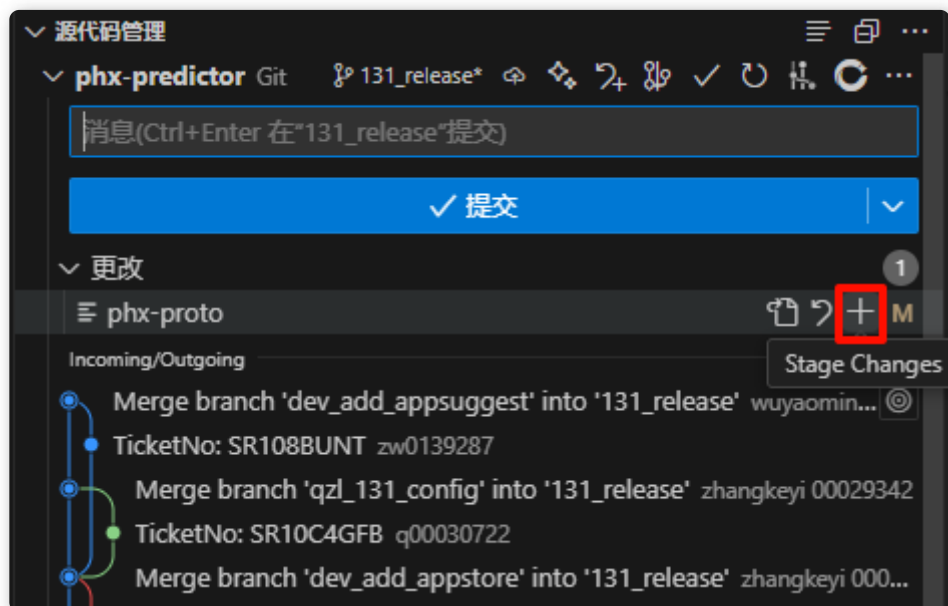
1. 把文件添加到暂存区：

```
git add [文件]
```

注意：添加之后行标和概览处就不会有修改的标记了，如下↓



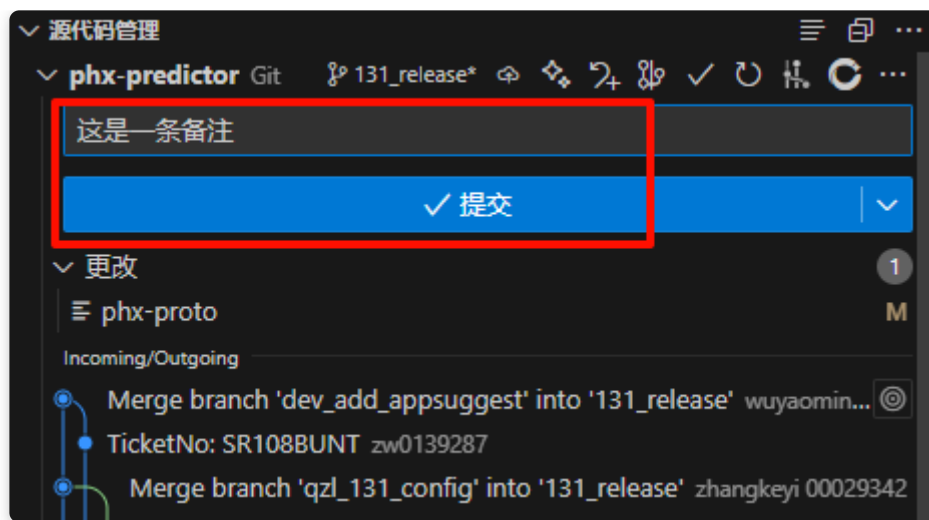
或者直接小手点一点：



2. 把暂存区的代码提交为分支新节点：

```
git commit 或 git commit -m "修改的备注：改了啥" (更推荐)
```

或者直接小手点一点：



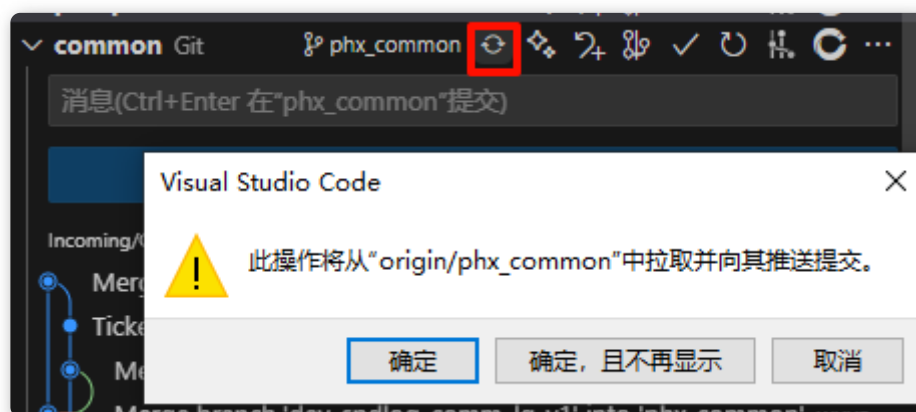
如果忘了写备注并且已经提交了或者想修改上次提交的备注：

`git commit --amend`，会打开默认的文本编辑器（vim），在里面添加或修改即可

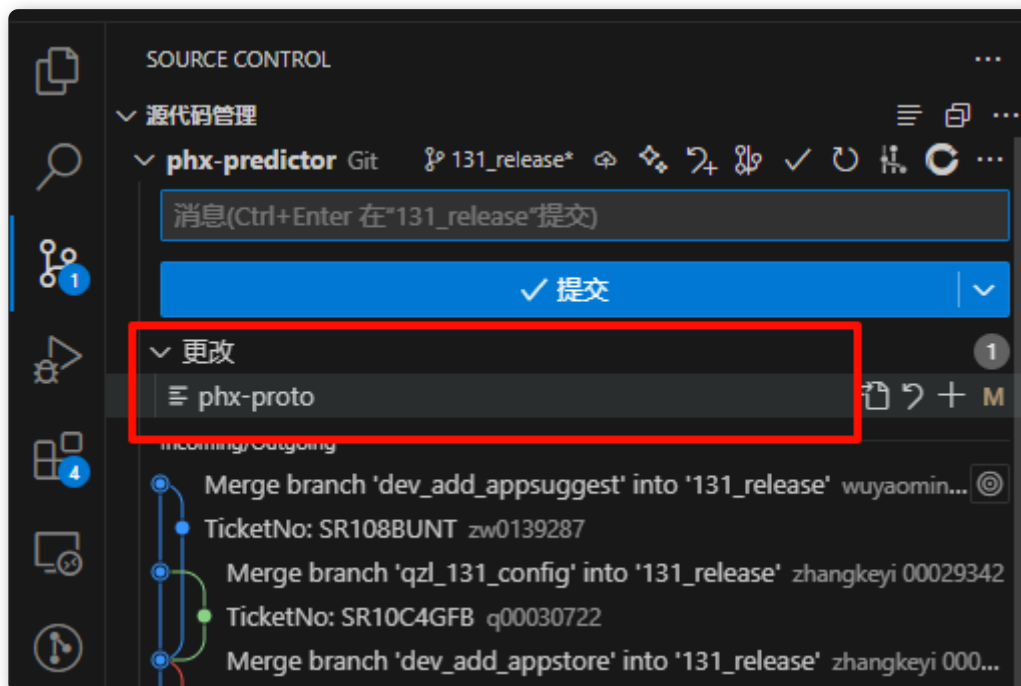
3. 拉取分支最新代码：

`git pull origin [分支名]`

或者直接小手点一点：



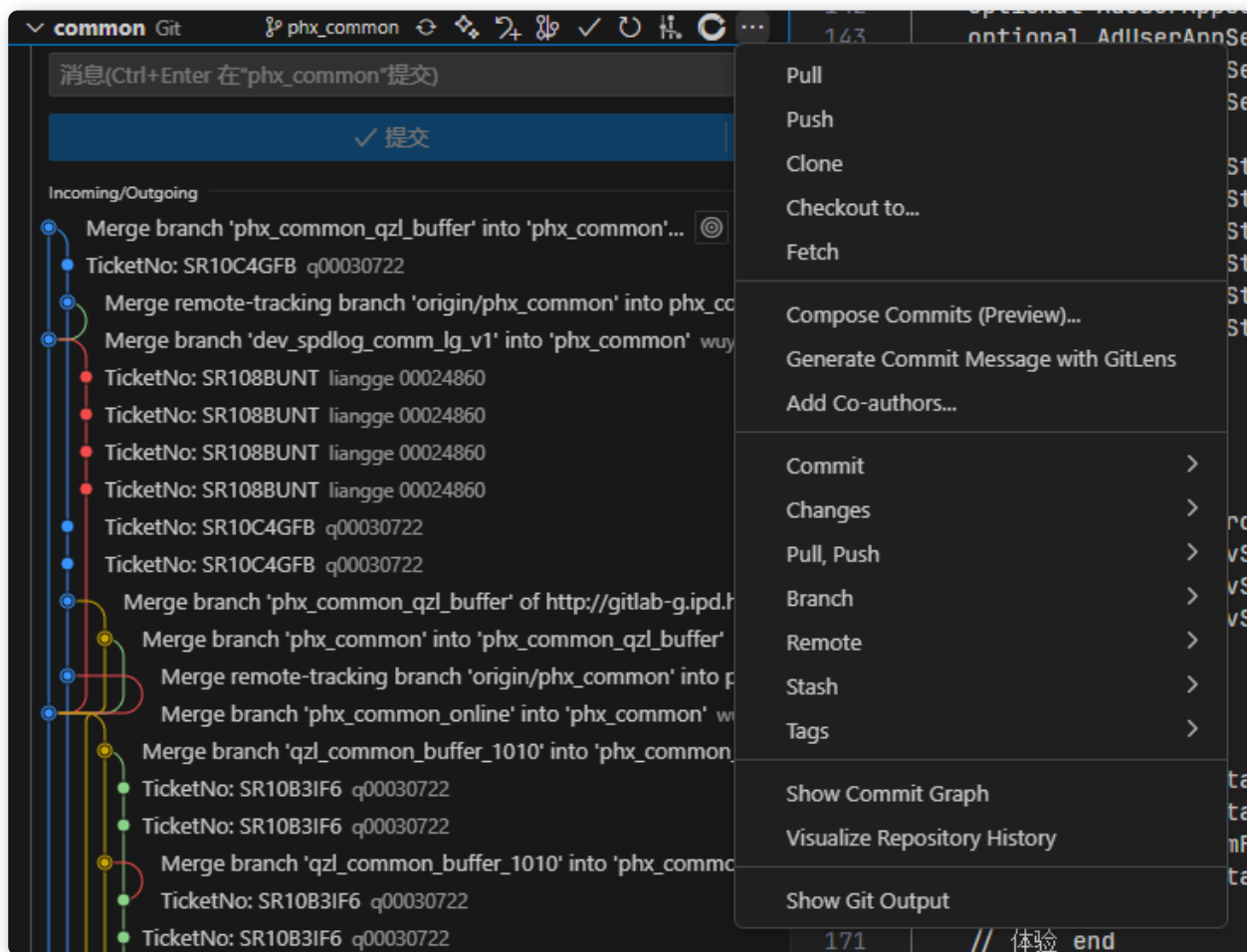
注：如果拉取的分支和当前分支有冲突，用fetch+merge，这时代码管理会显示需要你解冲突的文件，点进去会出现diff



4. 推送到远程仓库:

```
git push origin [分支名]
```

或者直接小手点一点:

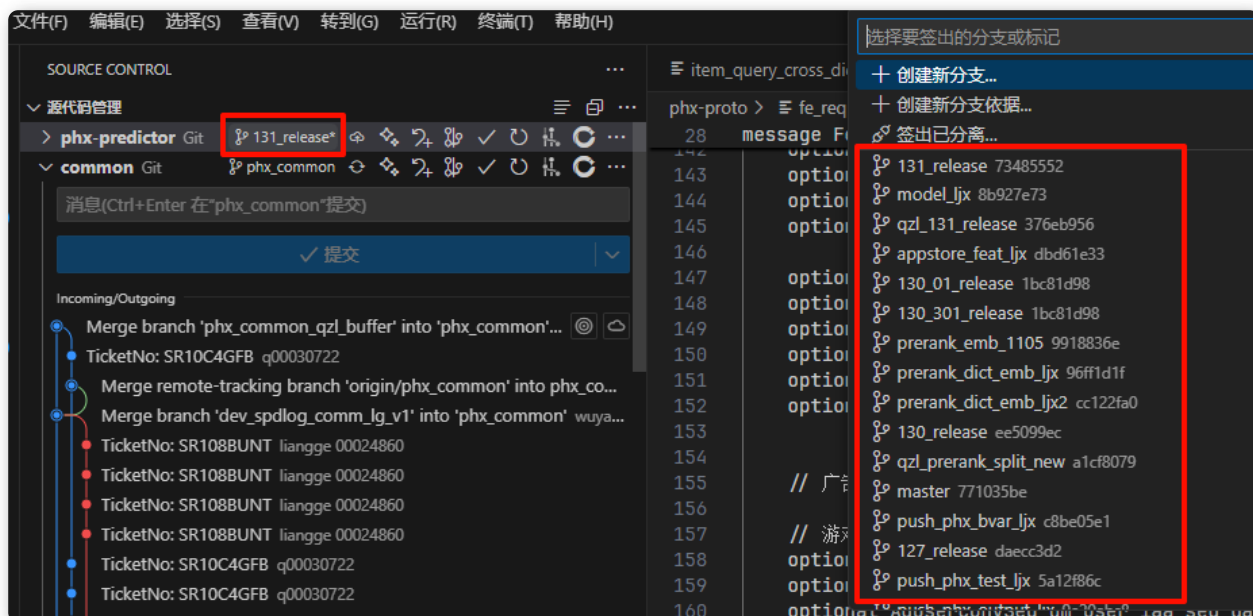


分支相关操作

1. 切换分支:

```
git checkout [分支]
```

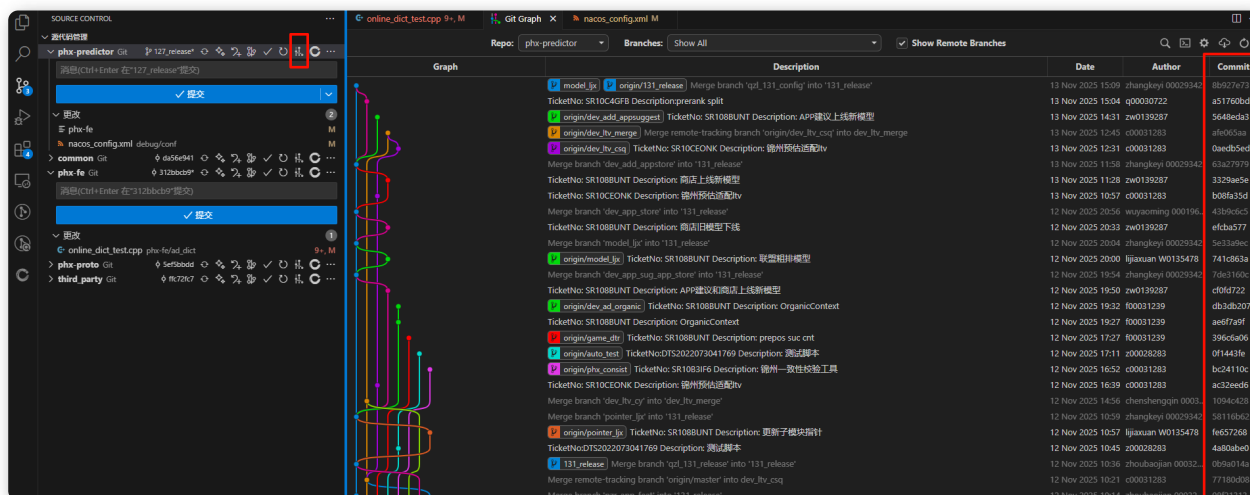
或者直接小手点一点:



2. 在当前分支的基础上拉一个新的分支并切换到新分支上：

```
git checkout -b [基于旧分支的新分支]
```

3. 返回分支里的某一个节点：



- 返回上一个节点：

```
git reset --soft HEAD^
```

- 返回指定节点：

```
git reset --soft [分支节点名]
```

- 一般返回之前的旧节点之后想推送会显示：你的分支落后于远程仓库的分支，这时候可以强制推送：

```
git push origin [分支名] --force
```

4. 给当前分支改名

```
git branch -m [分支名]
```

5. 删除分支（本地分支和推送到远程仓库的分支互不干扰）：

- 看本地所有已创建的分支：

```
git branch -v
```

- 删除本地分支

```
git branch -d [不要的分支]
```

- 删除远程分支

```
git push origin --delete [不要的分支]
```

储藏栈相关操作

正在一个分支上开发到一半，需要临时切换分支处理其他任务（比如修复紧急bug）但又不想提交未完成的代码时：

1. 将当前工作区和暂存区的改动保存到储藏栈中，并恢复干净的工作区：

```
git stash 或 git stash push -m "备注"
```

2. 查看储藏列表，最新的储藏是 stash@{0}:

```
git stash list
```

3. 恢复指定的储藏（默认是最新的 stash@{0}）到当前工作区

- 恢复之后从储藏栈中删除该储藏:

```
git stash pop 或 git stash pop [stash@{n}]
```

- 恢复之后不删除储藏栈中的记录:

```
git stash apply 或 git stash apply [stash@{n}]
```

4. 删除储存:

- 删除指定的储藏（默认是最新的 stash@{0}）:

```
git stash drop 或 git stash drop [stash@{n}]
```

- 清空整个储藏栈，一般不会用到:

```
git stash clear
```

5. 查看指定储藏的详细改动内容 (diff)

```
git stash show -p [stash@{n}]
```

其他

如果项目中嵌套了其他子项目，想强制更新所有子模块:

```
git submodule update --init --recursive
```

好用的vscode git工具插件

Git Graph、GitLens

